

Calcolatori Elettronici

18 giugno 2024

Teoria

Esercizio 1 (13 punti): Data una sequenza infinita di caratteri in ingresso appartenenti all'alfabeto "i, o, p", progettare la rete sequenziale che riconosca le occorrenze della sequenza "pippo", ipotizzando che l'alfabeto di uscita sia "sequenza riconosciuta, sequenza non riconosciuta". La rete deve essere in grado di riconoscere anche sequenze sovrapposte. Realizzare il circuito equivalente dell'equazione minimizzata di eccitazione di un singolo flip flop di stato.

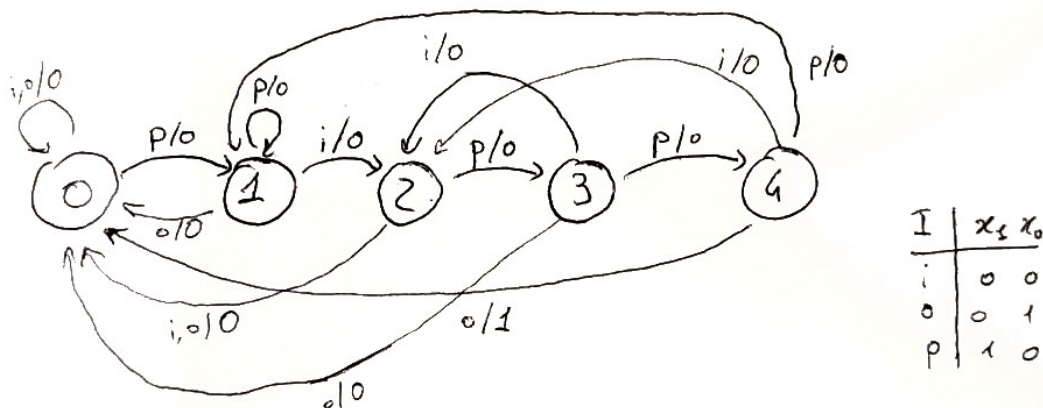
Esercizio 2 (11 punti): Si considerino i numeri $A=2X$ e $B=-3Y$, dove ad X e Y sono stati sostituiti, rispettivamente, il penultimo e l'ultimo numero della propria matricola. Convertire entrambi i numeri in complemento a due a 8 bit e calcolare $A+B$ mostrando tutti i passaggi.

Esercizio 3 (6 punti): Illustrare vantaggi e svantaggi della cache set associativa ad n vie confrontandola con la cache ad accesso diretto.

Soluzioni

Esercizio 1

L'automa a stati finiti che risolve il problema può essere il seguente:



in cui è indicata anche la codifica degli input. Per gli stati, si utilizza la normale codifica binaria. Per l'uscita, 1=sequenza riconosciuta, 0=sequenza non riconosciuta.

Si procede con la realizzazione della tabella degli stati e transizioni. Non interessa l'output, poiché viene richiesto soltanto l'equazione di eccitazione di un flip/flop di stato. Si ottiene:

y_2	y_1	y_0	x_1	x_0	y_2'	y_1'	y_0'
0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0
0	0	0	1	0	0	0	1
0	0	0	1	1	-	-	-
0	0	1	0	0	0	1	0
0	0	1	0	1	0	0	0
0	0	1	1	0	0	0	1
0	0	1	1	1	-	-	-
0	1	0	0	0	0	0	0
0	1	0	0	1	0	0	0
0	1	0	1	0	0	1	1
0	1	0	1	1	-	-	-
0	1	1	0	0	0	0	1
0	1	1	0	1	0	0	0
0	1	1	1	0	1	0	0
0	1	1	1	1	-	-	-
1	0	0	0	0	0	1	0
1	0	0	0	1	0	0	0
1	0	0	1	0	0	0	1
1	0	0	1	1	-	-	-

Si sceglie di studiare l'equazione di eccitazione di y_0' . Si realizza quindi la mappa di Karnaugh:

$y_2 y_1$	00	01	11	10
00	0	0	-	0
01	1	1	-	1
11	1	0	-	-
10	0	1	-	-

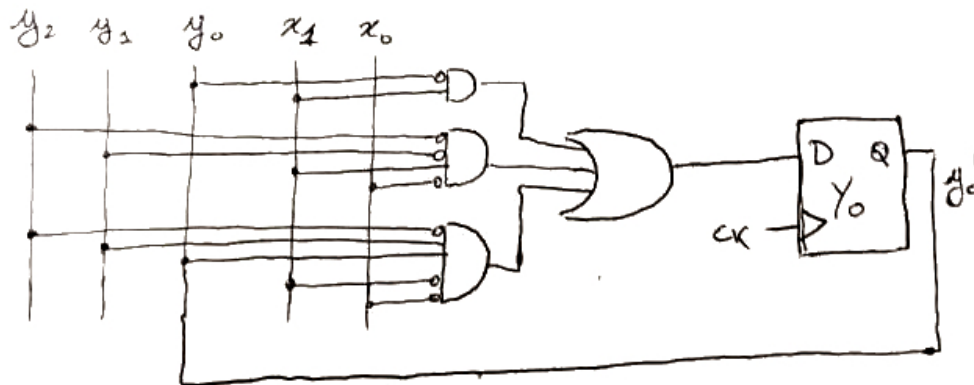
$$x_0 = 0$$

$y_2 y_1$	00	01	11	10
00	0	0	-	0
01	1	-	-	-
11	1	-	-	-
10	0	0	-	-

$$x_0 = 1$$

Da cui si ottiene l'equazione di eccitazione ed il circuito equivalente:

$$y_0' = \bar{y}_0 x_1 + \bar{y}_2 \bar{y}_1 x_1 \bar{x}_0 + \bar{y}_2 y_1 y_0 x_1 \bar{x}_0$$



Esercizio 2

Si suppongono $x=7$ e $y=5$. Da cui:

1. Convertiamo in binario: $27_{10} = 0001\ 1011_2$, utilizzando le divisioni successive. Tale numero è positivo, pertanto è già rappresentato in complemento a 2—il bit più significativo è 0.
2. Convertiamo 35_{10} in binario: $35_{10} = 0010\ 0011_2$, utilizzando le divisioni successive.
3. Invertiamo tutti i bit per ottenere il complemento a uno: Complemento a uno di $0010\ 0011_2 = 1101\ 1100_2$
4. Aggiungiamo 1 per ottenere il complemento a due: $1101\ 1100_2 + 1 = 1101\ 1101_2$
5. Sommiamo i due numeri in binario (ricordando di ignorare il riporto finale fuori dagli 8 bit):

0	0	0	1	1	0	1	1
1	1	0	1	1	1	0	1
0	0	1	1	1	0	1	0

 (ignora il riporto fuori dagli 8 bit)

Progetto

Un processore z64 governa un dispositivo in grado di effettuare un clone del contenuto di un disco rigido (**SORGENTE**) in un altro disco rigido (**DESTINAZIONE**). Entrambe le periferiche **SORGENTE** e **DESTINAZIONE** sono attestate sul DMAC di sistema. La periferica **PULSANTE** consente l'avvio della copia dei dati contenuti all'interno di **SORGENTE** verso **DESTINAZIONE** .

PULSANTE è una periferica asincrona. Alla pressione del tasto, lo z64 programma il DMAC di sistema per copiare da **SORGENTE** 1MB di dati in un buffer contenuto in memoria RAM. Al termine dell'acquisizione, lo z64 programma nuovamente il DMAC di sistema per trasferire il contenuto del buffer verso **DESTINAZIONE** . La quantità complessiva dei dati da copiare è pari a 1GB.

Fino a che il sistema non ha completato la copia di tutto il GB di dati, qualsiasi pressione di **PULSANTE** viene ignorata dal processore. Al termine della copia, una ulteriore pressione di **PULSANTE** permette di effettuare nuovamente la copia dei dati.

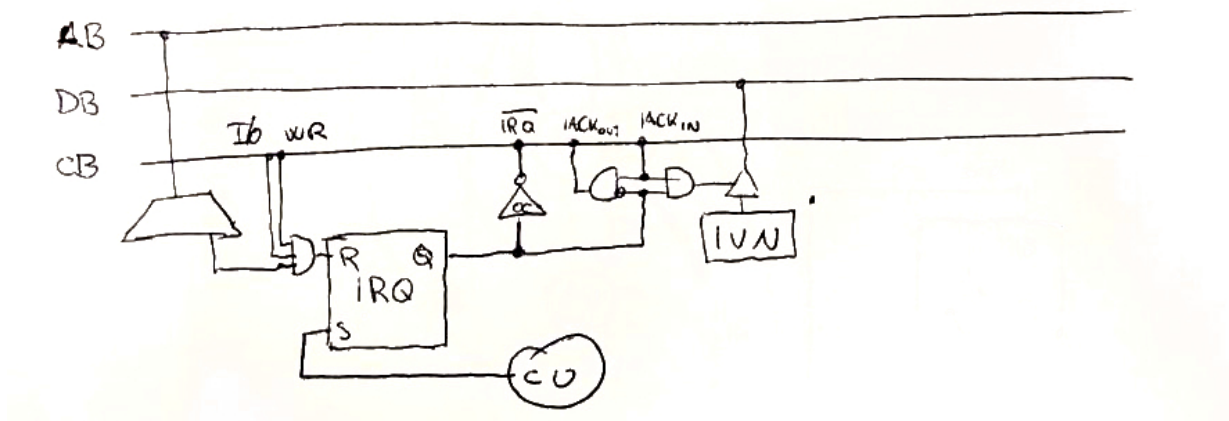
Realizzare:

- Le interfacce dei dispositivi **PULSANTE** e **SORGENTE** (**DESTINAZIONE** è molto simile a **SORGENTE** e non è necessario realizzarla);
- Tutto codice necessario al funzionamento del sistema, senza l'utilizzo di pseudo-istruzioni.

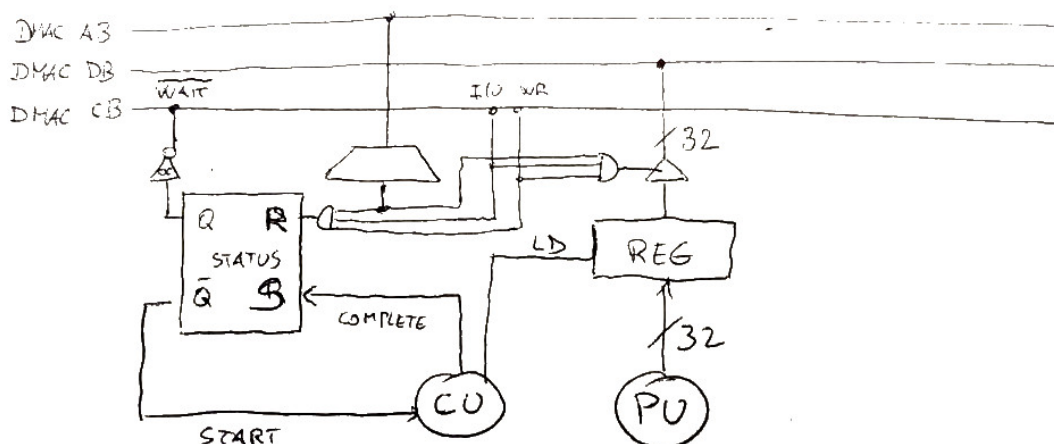
Soluzione

La seguente è una sola delle molte possibili soluzioni dell'esercizio. Si è assunto che la bobina alterni periodi di accensione e spegnimento della stessa durata. Qualsiasi altra interpretazione è ugualmente valida.

PULSANTE è una semplice interfaccia di una periferica che genera richieste di interruzione. Si decide di ignorare le richieste di interruzioni a software, quindi non si prevede un flip flop di status (che sarebbe stato comunque corretto).



SORGENTE è una periferica di input sincrona. Essendo attestata sul bus del DMAC, poiché si ha a disposizione un solo numero di porta per la comunicazione, il protocollo prevede l'esistenza di un segnale di blocco del DMAC (**WAIT** in questa soluzione, o **DEV_COMPLETE** secondo lo schema fornito a lezione).



Una possibile implementazione del software è la seguente.

```

1  .org 0x800
2  .data
3      buffer: .fill 1024
4      .equ SORGENTE, 0xabcd
5      .equ DESTINAZIONE, 0xbcde
6      .equ PULSANTE_IRQ, 0x0001
7      copy_in_progress: .byte 0 ; Flag per indicare se una copia è in corso
8
9  .section .text
10
11 main:
12     cmpb $1, copy_in_progress
13     jnz main
14     call start_copy
15     movb $0, copy_in_progress
16     jmp main
17
18 .driver 0
19     # Ignora interrupt request se una copia è già in corso
20     cmpb $0, copy_in_progress
21     jnz .end_irq ; Se copia in corso, esce dalla ISR
22     movb $1, copy_in_progress
23 .end_irq:
24     iret
25
26 start_copy:
27     cld
28     movq $buffer, %rdi
29     movq $buffer, %rsi
30     xorw %ax, %ax
31 .loop:
32     movw $SORGENTE, %dx
33     movq $1024*1024/4, %rcx # 1MB a blocchi di 32 bit
34     insl
35     movw $DESTINAZIONE, %dx
36     movq $1024*1024/4, %rcx # 1MB a blocchi di 32 bit
37     outsl
38     addw $1, %ax
39     cmpw $1024, %ax # 1024 volte 1MB = 1GB
40     jnz .loop
41     ret

```

Prova di laboratorio

Si desidera realizzare un programma in C che implementi una semplice cifratura/decifratura di stringhe di testo. Il programma accetta un argomento a riga di comando, che sarà la parola utilizzata come chiave di cifratura. Ad esempio, una invocazione valida del programma è la seguente:

```
1 | $ ./crypt chiave
```

Il programma funziona come segue. In primo luogo, viene calcolato il numero di bit impostati a 1 nella rappresentazione ASCII della chiave. Tale numero viene memorizzato in una variabile a 8 bit—in caso di overflow, il risultato rimane comunque corretto e utilizzabile. Questa variabile rappresenta la chiave di cifratura effettiva.

Successivamente, il programma richiede all'utente di inserire da tastiera una frase da cifrare o decifrare. A ciascun carattere della stringa inserita, viene applicata la chiave effettiva utilizzando l'operatore XOR, ovvero `input[i] XOR chiave_effettiva`.

La nuova stringa ottenuta viene stampata a schermo. Al termine di queste operazioni, il programma termina.

Soluzione

La seguente è una possibile soluzione all'esercizio.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  unsigned char conta_bit_impostati(const char *chiave) {
6      unsigned char count = 0;
7      for (size_t i = 0; i < strlen(chiave); ++i) {
8          unsigned char c = chiave[i];
9          while (c) {
10             count += c & 1;
11             c >>= 1;
12         }
13     }
14     return count;
15 }
16
17 int main(int argc, char *argv[]) {
18     if (argc != 2) {
19         fprintf(stderr, "Uso: %s <chiave>\n", argv[0]);
20         return 1;
21     }
22
23     unsigned char chiave_cifratura = conta_bit_impostati(argv[1]);
24     char input[256];
25     printf("Inserisci una frase da cifrare/decifrare: ");
26     if (fgets(input, sizeof(input), stdin) == NULL) {
27         fprintf(stderr, "Errore nella lettura dell'input\n");
28         return 1;
29     }
30     size_t len = strlen(input);
31     if (input[len - 1] == '\n') {
32         input[len - 1] = '\0';
33         --len;
34     }
35
36     // Applichiamo la chiave di cifratura a ciascun carattere della stringa e stampiamo
37     for (size_t i = 0; i < len; ++i)
38         input[i] ^= chiave_cifratura;
39     printf("Risultato: %s\n", input);
40
41     return 0;
42 }
```

Per verificare l'esecuzione si può testare il funzionamento del programma come segue:

```
1  $ ./a.out asd
2  Inserisci una frase da cifrare/decifrare: supercalifragilistichepiralidoso
3  Risultato: x~{nyhjgbmyjlbgbxbhcnx{byjgbodxd
4  $ ./a.out asd
5  Inserisci una frase da cifrare/decifrare: x~{nyhjgbmyjlbgbxbhcnx{byjgbodxd
6  Risultato: supercalifragilisichespiralidoso
```